

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



SWE 4601 - SOFTWARE DESIGN AND ARCHITECTURES

Quiz 3 - VIPER Architecture pattern implementation

Written by - Namisa Najah Raisa

Student ID: 210042112

Sixth Semester

Department of Computer Science and Engineering
BSc. in Software Engineering

October 11, 2025

Abstract

This report presents a comprehensive analysis of the Movie Journal application, a React Native mobile application built using the VIPER (View, Interactor, Presenter, Entity, Router) architectural pattern. The application enables users to create, manage, and track their movie reviews with Firebase backend integration. This document provides a detailed justification for the VIPER architecture adoption and examines each component's role in maintaining separation of concerns, testability, and scalability.

Contents

1	Executive Summary	5
1.1	Key Features	5
1.2	Technology Stack	5
2	VIPER Architecture Overview	5
2.1	What is VIPER?	5
2.2	Why VIPER for This Project?	5
2.2.1	Separation of Concerns	6
2.2.2	Testability	6
2.2.3	Scalability	6
2.2.4	Team Collaboration	6
2.2.5	Reusability	6
3	Entity Layer: Business Objects	6
3.1	Purpose and Responsibility	6
3.2	ReviewEntity Implementation	6
3.3	Business Logic Encapsulation	7
3.3.1	Validation Logic	7
3.3.2	Presentation Helpers	7
3.3.3	Data Transformation	7
3.4	Justification	7
4	Interactor Layer: Business Logic	7
4.1	Purpose and Responsibility	7
4.2	Module-Specific Interactors	8
4.2.1	AddReviewInteractor	8
4.2.2	HomeInteractor	8
4.2.3	AuthInteractor	9
4.2.4	ProfileInteractor	9
4.3	Justification	10
5	Presenter Layer: Presentation Logic	10
5.1	Purpose and Responsibility	10
5.2	Module-Specific Presenters	11
5.2.1	AddReviewPresenter	11
5.2.2	HomePresenter	11
5.2.3	AuthPresenter	12
5.2.4	ProfilePresenter	12
5.3	Justification	13
6	View Layer: User Interface	13
6.1	Purpose and Responsibility	13
6.2	Module-Specific Views	13
6.2.1	AddReviewView	13
6.2.2	HomeView	14
6.2.3	AuthView & RegisterView	14

6.2.4	ProfileView	15
6.3	Reusable Components	15
6.3.1	AnimatedButton	15
6.3.2	FloatingHearts	15
6.3.3	PastelBanner	15
6.3.4	PastelConfirmModal	15
6.4	Justification	15
7	Router Layer: Navigation	16
7.1	Purpose and Responsibility	16
7.2	Module-Specific Routers	16
7.2.1	AddReviewRouter	16
7.2.2	HomeRouter	16
7.2.3	AuthRouter	17
7.2.4	ProfileRouter	17
7.3	Navigation Structure	17
7.4	Justification	18
8	Data Flow in VIPER	18
8.1	Complete User Action Flow	18
8.2	Data Flow Diagram	19
9	Benefits of VIPER in This Project	19
9.1	Achieved Benefits	19
9.1.1	Separation of Concerns	19
9.1.2	Testability	19
9.1.3	Maintainability	20
9.1.4	Scalability	20
9.1.5	Team Collaboration	20
9.2	Comparison with Alternative Architectures	20
10	Challenges and Solutions	21
10.1	Challenges Faced	21
10.1.1	Increased Boilerplate	21
10.1.2	Over-Engineering for Simple Features	21
10.2	Best Practices Implemented	21
11	Future Enhancements	21
11.1	Potential Features	21
11.1.1	Social Features Module	21
11.1.2	Search Module	22
11.1.3	External API Integration	22
11.2	Architecture Benefits for Extensions	22
12	Conclusion	22
12.1	Key Takeaways	22

1 Executive Summary

The Movie Journal application "**Diary of a cinephile**" is a personal movie review management system developed using **React Native** and **Firebase**. The application implements the VIPER architectural pattern to achieve a clean separation of concerns, enhanced testability, and a maintainable code structure.

1.1 Key Features

- User authentication (login and registration)
- Create, read, update, and delete movie reviews
- Rating system (1-5 stars)
- Personal profile with statistics
- Cloud-based data persistence with Firebase

1.2 Technology Stack

- **Framework:** React Native
- **Navigation:** React Navigation
- **Backend:** Firebase (Firestore, Authentication)
- **Architecture:** VIPER Pattern

2 VIPER Architecture Overview

2.1 What is VIPER?

VIPER is an architectural pattern that stands for:

- **View** - User interface layer
- **Interactor** - Business logic layer
- **Presenter** - Presentation logic layer
- **Entity** - Data model layer
- **Router** - Navigation logic layer

2.2 Why VIPER for This Project?

The Movie Journal application adopts VIPER architecture for the following strategic reasons:

2.2.1 Separation of Concerns

Each component has a single, well-defined responsibility, making the codebase more maintainable and easier to understand. This is particularly important as the application handles multiple complex operations including authentication, data persistence, and UI state management.

2.2.2 Testability

VIPER's modular structure allows for comprehensive unit testing. Each layer can be tested independently:

- Interactors can be tested without UI
- Presenters can be tested with mock interactors
- Views can be tested with mock presenters

2.2.3 Scalability

As the application grows, new features can be added as new modules without affecting existing functionality. The clear boundaries between components prevent feature creep and maintain code quality.

2.2.4 Team Collaboration

Different team members can work on different layers simultaneously without conflicts. A designer can work on Views while a backend developer implements Interactors.

2.2.5 Reusability

Business logic in Interactors can be reused across different Views. For example, the review validation logic is centralized and can be used by multiple screens.

3 Entity Layer: Business Objects

3.1 Purpose and Responsibility

The Entity layer contains business objects that represent the data model and encapsulate business logic. Entities are plain data structures with methods that define business rules.

3.2 ReviewEntity Implementation

The ReviewEntity class serves as the core business object:

```
1 export default class ReviewEntity {
2   constructor(data = {}) {
3     this.id = data.id || null;
4     this.movieTitle = data.movieTitle || '';
5     this.review = data.review || '';
6     this.rating = data.rating || 0;
7     this.posterUrl = data.posterUrl || '';
```

```
8     this.date = data.date || new Date().toISOString();
9     this.userId = data.userId || null;
10  }
11
12  // Business logic methods
13  isValid() { ... }
14  getStars() { ... }
15  getFormattedDate() { ... }
16  getReviewPreview(maxLength) { ... }
17 }
```

Listing 1: ReviewEntity Structure

3.3 Business Logic Encapsulation

3.3.1 Validation Logic

The `isValid()` method encapsulates review validation rules:

- Movie title must not be empty
- Rating must be between 1 and 5

This ensures consistent validation across the application, preventing invalid data from reaching the database.

3.3.2 Presentation Helpers

Methods like `getStars()` and `getFormattedDate()` handle data formatting, keeping presentation logic separate from the View layer while maintaining consistency.

3.3.3 Data Transformation

The `toFirebaseObject()` method transforms the entity into a Firebase-compatible format, isolating Firebase-specific concerns from the rest of the application.

3.4 Justification

Why Entity Layer is Essential:

1. **Single Source:** All business rules are defined in one place
2. **Type Safety:** Provides a consistent data structure throughout the app
3. **Business Logic Centralization:** Validation and formatting logic is reusable
4. **Framework Independence:** Entities don't depend on React Native or Firebase

4 Interactor Layer: Business Logic

4.1 Purpose and Responsibility

The Interactor layer handles all business logic and data operations. It communicates with external services (Firebase), performs data validation, and implements use cases.

4.2 Module-Specific Interactors

4.2.1 AddReviewInteractor

Handles review creation and updates:

```
1 export default class AddReviewInteractor {
2   async saveReview(reviewEntity) {
3     const user = auth.currentUser;
4     if (!user) throw new Error("No user logged in");
5
6     if (!reviewEntity.isValid()) {
7       throw new Error("Invalid review data");
8     }
9
10    reviewEntity.userId = user.uid;
11    const payload = reviewEntity.toFirebaseObject();
12    const docRef = await addDoc(this.collectionRef, payload);
13    return docRef.id;
14  }
15 }
```

Listing 2: AddReviewInteractor Structure

Responsibilities:

- Authenticate user before operations
- Validate review data using Entity methods
- Transform entities to Firebase format
- Handle database operations
- Return operation results

4.2.2 HomeInteractor

Manages review retrieval and deletion:

```
1 async fetchReviews() {
2   const user = auth.currentUser;
3   if (!user) return [];
4
5   const q = query(
6     this.collectionRef,
7     where("userId", "==", user.uid),
8     orderBy("date", "desc")
9   );
10
11   const snapshot = await getDocs(q);
12   return snapshot.docs.map(
13     doc => ReviewEntity.fromFirebaseDoc(doc)
14   );
15 }
```

Listing 3: HomeInteractor Operations

Responsibilities:

- Filter reviews by authenticated user
- Order reviews chronologically
- Transform Firebase documents to ReviewEntity instances
- Handle deletion operations

4.2.3 AuthInteractor

Manages authentication operations:

```
1 async login(email, password) {
2   const user = await signInWithEmailAndPassword(
3     auth, email, password
4   );
5   return user.user;
6 }
7
8 async register(email, password, displayName) {
9   const cred = await createUserWithEmailAndPassword(
10    auth, email, password
11  );
12
13  if (displayName) {
14    await updateProfile(cred.user, { displayName });
15  }
16
17  // Create user document in Firestore
18  await setDoc(doc(db, "users", cred.user.uid), {
19    email: cred.user.email,
20    displayName: displayName || null,
21    createdAt: serverTimestamp(),
22  });
23
24  return cred.user;
25 }
```

Listing 4: AuthInteractor Methods

Responsibilities:

- Handle Firebase authentication
- Create user profile documents
- Manage authentication state
- Handle authentication errors

4.2.4 ProfileInteractor

Manages user statistics and profile data:

```
1 async fetchAllReviews() {
2   const user = auth.currentUser;
3   if (!user) return [];
4   const q = query(this.collectionRef, where('userId', '==', user.uid)
5 );
```

```
5     const snapshot = await getDocs(q);
6     return snapshot.docs.map(doc => ReviewEntity.fromFirestoreDoc(doc));
7 }
8
9 async logout() {
10     await signOut(auth);
11     return true;
12 }
13
14 async fetchUserInfo() {
15     const user = auth.currentUser;
16     if (!user) return null;
17     const userDoc = await getDoc(doc(db, 'users', user.uid));
18     const profile = userDoc.exists() ? userDoc.data() : {};
19     return {
20         uid: user.uid,
21         email: user.email,
22         displayName: user.displayName || profile.displayName || null,
23         createdAt: profile.createdAt || null,
24     };
25 }
```

Listing 5: AuthInteractor Methods

Responsibilities:

- Fetch all user reviews for statistics
- Retrieve user profile information
- Handle logout operations

4.3 Justification

Why Interactor Layer is Critical:

1. **Business Logic Isolation:** All Firebase operations are contained here
2. **Reusability:** Logic can be used by multiple Presenters
3. **Testability:** Can be tested without UI or network
4. **Error Handling:** Centralized error management
5. **Security:** User authentication checks before operations

5 Presenter Layer: Presentation Logic

5.1 Purpose and Responsibility

The Presenter acts as the middleman between View and Interactor. It receives user actions from the View, coordinates with the Interactor, and prepares data for display.

5.2 Module-Specific Presenters

5.2.1 AddReviewPresenter

Handles review submission logic:

```
1 async submitReview(data) {
2   // Create ReviewEntity from raw data
3   const entity = new ReviewEntity({
4     movieTitle: data.movieTitle,
5     review: data.review,
6     rating: Number(data.rating),
7     posterUrl: data.posterUrl,
8     date: new Date().toISOString(),
9   });
10
11   // Validate using entity's business logic
12   if (!entity.isValid()) {
13     throw new Error('Title and rating (1-5) required');
14   }
15
16   // Determine save or update
17   if (data.id) {
18     await this.interactor.updateReview(data.id, entity);
19     return data.id;
20   } else {
21     return await this.interactor.saveReview(entity);
22   }
23 }
```

Listing 6: AddReviewPresenter Logic

Responsibilities:

- Transform View data into Entities
- Coordinate save vs. update operations
- Handle validation errors
- Return success/failure status

5.2.2 HomePresenter

Transforms data for display:

```
1 async getReviewsForView() {
2   const entities = await this.interactor.fetchReviews();
3
4   // Transform entities into view-ready data
5   return entities.map((entity) => ({
6     id: entity.id,
7     movieTitle: entity.movieTitle,
8     review: entity.review,
9     rating: entity.rating,
10    stars: entity.getStars(), // Use entity logic
11    posterUrl: entity.posterUrl,
12    date: entity.getFormattedDate(), // Use entity logic
13  }));
14 }
```

14 }

Listing 7: HomePresenter Data Transformation

Responsibilities:

- Fetch data from Interactor
- Transform ReviewEntity instances to view-friendly format
- Utilize Entity methods for presentation (stars, formatted date)
- Coordinate delete operations

5.2.3 AuthPresenter

Coordinates authentication:

```
1 async handleLogin(email, password) {
2   return await this.interactor.login(email, password);
3 }
4
5 async handleRegister(email, password, displayName) {
6   return await this.interactor.register(email, password, displayName)
7   ;
8 }
```

Listing 8: ProfilePresenter Statistics

Responsibilities:

- Delegate login/register to Interactor
- Handle authentication responses
- Prepare error messages for View

5.2.4 ProfilePresenter

Calculates user statistics:

```
1 async getStats() {
2   const reviews = await this.interactor.fetchAllReviews();
3   const total = reviews.length;
4   const avg = total === 0 ? 0 :
5     reviews.reduce((s, r) => s + Number(r.rating), 0) / total;
6
7   return {
8     total,
9     averageRating: avg
10  };
11 }
12
13 async logout() {
14   return await this.interactor.logout();
15 }
16
17 async getUserInfo() {
18   return await this.interactor.fetchUserInfo();
19 }
```

Listing 9: ProfilePresenter Statistics

Responsibilities:

- Calculate aggregate statistics
- Format user information
- Coordinate logout operation

5.3 Justification

Why Presenter Layer is Essential:

1. **View Independence:** View doesn't know about business logic
2. **Data Transformation:** Converts complex entities to simple view models
3. **Coordination:** Orchestrates multiple Interactor calls
4. **Testability:** Can be tested with mock Interactors
5. **Flexibility:** Same Presenter can support different Views

6 View Layer: User Interface

6.1 Purpose and Responsibility

The View layer handles all UI rendering and user interactions. It displays data prepared by the Presenter and forwards user actions back to the Presenter.

6.2 Module-Specific Views

6.2.1 AddReviewView

Provides interface for creating/editing reviews:

Key Features:

- Form inputs for movie details
- Validation feedback via banners
- Pre-populated fields for editing

View Responsibilities:

- Render form controls
- Manage local UI state (input values)
- Call Presenter's `submitReview`
- Display success/error feedback
- Trigger Router navigation

6.2.2 HomeView

Displays list of user's reviews:

Key Features:

- FlatList with cards
- Edit and delete buttons per review
- Confirmation modal for deletions
- Loading states
- Empty state messaging

View Responsibilities:

- Render review list
- Handle edit/delete actions
- Manage confirmation modal state
- Display loading indicators
- Trigger navigation to add/edit screens

6.2.3 AuthView & RegisterView

Authentication interfaces:

Key Features:

- Email and password inputs
- Containers
- Error banner display
- Navigation between login/register

View Responsibilities:

- Render authentication forms
- Manage input state
- Call Presenter authentication methods
- Display error messages
- Navigate on successful auth

6.2.4 ProfileView

User profile and statistics:

Key Features:

- User information display
- Statistics cards (total reviews, average rating)
- Logout button
- Animated transitions

View Responsibilities:

- Display user data
- Render statistics
- Handle logout action
- Navigate to home or login

6.3 Reusable Components

6.3.1 AnimatedButton

Provides consistent button animations across the app with spring physics.

6.3.2 FloatingHearts

Success feedback animation showing floating bubbles.

6.3.3 PastelBanner

Consistent error/success message display with themed colors.

6.3.4 PastelConfirmModal

Reusable confirmation dialog for destructive actions.

6.4 Justification

Why View Layer Separation is Important:

1. **UI Focus:** Views only handle rendering and user input
2. **No Business Logic:** All logic delegated to Presenter
3. **Reusability:** Components can be reused across modules
4. **Easy Updates:** UI changes don't affect business logic
5. **Designer Friendly:** Designers can work on Views independently

7 Router Layer: Navigation

7.1 Purpose and Responsibility

The Router layer handles all navigation logic, including screen transitions, parameter passing, and navigation stack management.

7.2 Module-Specific Routers

7.2.1 AddReviewRouter

```
1 export default class AddReviewRouter {  
2   constructor(navigation) {  
3     this.navigation = navigation;  
4   }  
5  
6   goHome() {  
7     this.navigation.navigate("Home");  
8   }  
9 }
```

Listing 10: AddReviewRouter

Responsibilities:

- Navigate back to Home after save

7.2.2 HomeRouter

```
1 goToAddReview() {  
2   this.navigation.navigate("AddReview");  
3 }  
4  
5 goToProfile() {  
6   this.navigation.navigate("Profile");  
7 }  
8  
9 goToEditReview(review) {  
10   this.navigation.navigate("AddReview", { review });  
11 }
```

Listing 11: HomeRouter Navigation

Responsibilities:

- Navigate to add review screen
- Navigate to profile screen
- Navigate to edit review with parameters

7.2.3 AuthRouter

```
1 goToHome() {  
2   this.navigation.replace("Home");  
3 }  
4  
5 goToRegister() {  
6   this.navigation.navigate("Register");  
7 }  
8  
9 goToLogin() {  
10  this.navigation.replace("Login");  
11 }
```

Listing 12: AuthRouter Navigation

Responsibilities:

- Navigate to home (replacing auth screens)
- Switch between login and register
- Return to login on logout

7.2.4 ProfileRouter

```
1 goHome() {  
2   this.navigation.navigate('Home');  
3 }  
4  
5 goToLogin() {  
6   this.navigation.replace('Login');  
7 }
```

Listing 13: AuthRouter Navigation

Responsibilities:

- Navigate to home screen
- Navigate to login (on logout)

7.3 Navigation Structure

The app uses React Navigation with a stack navigator:

- **Initial Route:** Login
- **Auth Flow:** Login → Register
- **Main Flow:** Home → AddReview → Profile
- **Edit Flow:** Home → AddReview (with params)

7.4 Justification

Why Router Layer is Necessary:

1. **Navigation Logic Isolation:** All routing in one place
2. **Testability:** Navigation can be tested independently
3. **Flexibility:** Easy to change navigation library
4. **Centralized Control:** Navigation patterns consistent
5. **Parameter Management:** Clean parameter passing between screens

8 Data Flow in VIPER

8.1 Complete User Action Flow

Let's trace a complete user action: *Adding a new movie review*

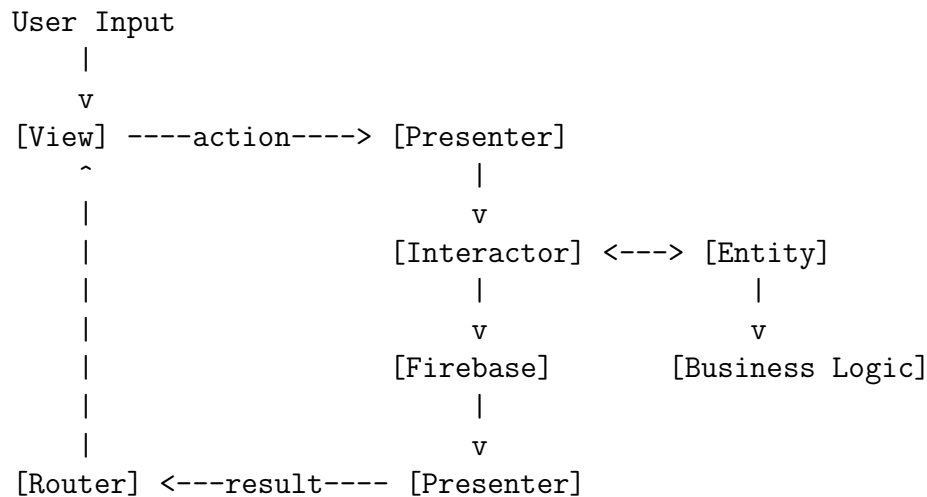
1. **User Action (View):** User fills form and taps "Save Review"
2. **View Layer:**
 - AddReviewView captures input data
 - Calls `presenter.submitReview(data)`
 - Shows loading state
3. **Presenter Layer:**
 - AddReviewPresenter receives raw data
 - Creates ReviewEntity from data
 - Validates using `entity.isValid()`
 - Calls `interactor.saveReview(entity)`
4. **Interactor Layer:**
 - AddReviewInteractor checks authentication
 - Validates entity again
 - Adds `userId` to entity
 - Transforms entity to Firebase format
 - Calls Firebase `addDoc()`
 - Returns document ID
5. **Response Flow (Interactor → Presenter → View):**
 - Interactor returns success with ID
 - Presenter returns ID to View
 - View shows success animation

- View calls `router.goHome()`

6. Router Layer:

- Router navigates to Home screen
- Home screen loads updated reviews

8.2 Data Flow Diagram



9 Benefits of VIPER in This Project

9.1 Achieved Benefits

9.1.1 Separation of Concerns

Each layer has a distinct responsibility:

- Views don't know about Firebase
- Interactors don't know about UI
- Entities contain only business logic
- Routers handle only navigation

9.1.2 Testability

Each component can be tested independently:

- Entities: Test validation, formatting logic
- Interactors: Test with mock Firebase
- Presenters: Test with mock Interactors
- Views: Test with mock Presenters
- Routers: Test navigation logic

9.1.3 Maintainability

- Clear code organization by feature
- Easy to locate bugs
- Simple to add new features
- Minimal impact from changes

9.1.4 Scalability

The architecture supports growth:

- Add new modules without modifying existing ones
- Reuse Interactors across multiple Views
- Share Entities across modules
- Extend functionality without breaking changes

9.1.5 Team Collaboration

Multiple developers can work simultaneously:

- Frontend developer works on Views
- Backend developer implements Interactors
- QA writes tests for Presenters
- Designer updates UI components

9.2 Comparison with Alternative Architectures

Aspect	MVC	MVP	VIPER
Separation of Concerns	Medium	Good	Excellent
Testability	Low	Good	Excellent
Learning Curve	Easy	Medium	Steep
Boilerplate Code	Low	Medium	High
Scalability	Medium	Good	Excellent

Table 1: Architecture Comparison

10 Challenges and Solutions

10.1 Challenges Faced

10.1.1 Increased Boilerplate

Challenge: VIPER requires more files per feature compared to simpler architectures.

Solution:

- Clear naming conventions
- Module-based organization
- Reusable base classes
- Benefits outweigh initial setup cost

10.1.2 Over-Engineering for Simple Features

Challenge: Some simple screens might not need full VIPER.

Solution:

- Maintain consistency across all features
- Future-proof for feature expansion
- Easier onboarding with consistent structure

10.2 Best Practices Implemented

1. **Consistent Module Structure:** Every feature follows the same pattern
2. **Single Responsibility:** Each class has one clear purpose
3. **Dependency Injection:** Interactors and Routers are injected into Presenters
4. **Error Handling:** Consistent error propagation through layers
5. **Async Operations:** Proper async/await usage throughout

11 Future Enhancements

11.1 Potential Features

11.1.1 Social Features Module

- Share reviews with friends
- Follow other users
- New ShareInteractor for social logic
- New SocialPresenter for feed management

11.1.2 Search Module

- Search reviews by title or rating
- Filter and sort options
- SearchInteractor for query logic
- SearchPresenter for result formatting

11.1.3 External API Integration

- Fetch movie posters from TMDB API
- Auto-complete movie titles
- MovieAPIInteractor for external data

11.2 Architecture Benefits for Extensions

The VIPER architecture makes these additions straightforward:

- New modules don't affect existing code
- Can reuse ReviewEntity across all modules
- Existing Interactors can be composed
- Testing remains independent per module

12 Conclusion

The Movie Journal application successfully demonstrates the implementation of VIPER architecture in a React Native project. Despite the initial complexity and boilerplate, the architecture provides significant benefits in terms of maintainability, testability, and scalability.

12.1 Key Takeaways

1. **Clear Separation:** Each layer has well-defined responsibilities
2. **Business Logic Protection:** Entities and Interactors safeguard business rules
3. **UI Independence:** Views are completely decoupled from business logic
4. **Navigation Management:** Routers provide clean navigation handling
5. **Future-Ready:** Architecture supports unlimited feature expansion

Appendix A: File Structure

```
src/  
  components/           # Reusable UI components  
    AnimatedButton.js  
    FloatingHearts.js  
    PastelBanner.js  
    PastelConfirmModal.js  
  entities/             # Business objects  
    ReviewEntity.js  
  firebase/             # Firebase configuration  
    firebaseConfig.js  
  modules/              # Feature modules (VIPER)  
    AddReview/  
      AddReviewView.js  
      AddReviewPresenter.js  
      AddReviewInteractor.js  
      AddReviewRouter.js  
    Auth/  
      AuthView.js  
      RegisterView.js  
      AuthPresenter.js  
      AuthInteractor.js  
      AuthRouter.js  
    Home/  
      HomeView.js  
      HomePresenter.js  
      HomeInteractor.js  
      HomeRouter.js  
    Profile/  
      ProfileView.js  
      ProfilePresenter.js  
      ProfileInteractor.js  
      ProfileRouter.js  
  navigation/  
    AppNavigator.js
```

Appendix B: VIPER Layer Responsibilities Summary

View Layer

- Renders UI components
- Captures user input
- Displays data from Presenter
- Manages local UI state (animations, modals)

- Calls Presenter methods for business operations
- Uses Router for navigation
- **Does NOT:** Contain business logic, call Interactors directly, manage navigation logic

Interactor Layer

- Implements business logic
- Performs network/database operations
- Manages data persistence (Firebase)
- Validates data using Entity methods
- Handles authentication checks
- Returns raw data or Entities
- **Does NOT:** Know about UI, perform navigation, format data for display

Presenter Layer

- Receives user actions from View
- Coordinates with Interactor
- Transforms Entities to view-friendly format
- Handles presentation logic
- Manages error states
- Returns formatted data to View
- **Does NOT:** Contain business logic, perform database operations, manage UI state

Entity Layer

- Defines data models
- Contains business validation logic
- Provides data transformation methods
- Encapsulates business rules
- Framework-independent
- **Does NOT:** Depend on UI, know about database, handle network operations

Router Layer

- Manages screen navigation
- Handles navigation parameters
- Controls navigation stack
- Provides navigation methods to View
- **Does NOT:** Contain business logic, manage data, render UI

Appendix C: Security Considerations

Authentication Security

- User authentication checked in all Interactors
- Firebase rules enforce user-specific data access
- No sensitive data stored in Views
- Secure token management by Firebase SDK

Data Validation

- Client-side validation in ReviewEntity
- Server-side validation via Firebase rules
- Input sanitization for text fields
- Rating constraints (1-5 range)

Nota Bene

For someone who wants to run the project but doesn't have firebase credentials, add this in the `firebaseConfig.js` file:

```
1 const firebaseConfig = {  
2   apiKey: "AIzaSyB7hb2rt9YfXH6rFI1YPCRC_8Rqjw3PDaE",  
3   authDomain: "sda-moviejournal.firebaseio.com",  
4   projectId: "sda-moviejournal",  
5   storageBucket: "sda-moviejournal.firebaseio.com",  
6   messagingSenderId: "1074245732163",  
7   appId: "1:1074245732163:web:ca6480d347ea7f5a67d1f6",  
8   measurementId: "G-MV46FWSTP9",  
9 };
```